

Univerzitet u Beogradu
Mašinski fakultet
Računarska inteligencija

$(1+(\lambda,\lambda))$ genetički algoritam za problem
trgovačkog putnika

Ognjen Spasojević 4015/2024

2026.

Sadržaj

Spisak slika	2
Spisak tabela	2
1 Uvod	3
2 Opis rešenja	3
2.1 Funkcija cilja	3
2.2 Genetski algoritam	4
2.3 $(1 + (\lambda, \lambda))$ genetički algoritam	4
2.4 Pošteno poređenje	4
3 Eksperimentalni rezultati	5
3.1 Eksperimentalno okruženje	5
3.2 Korišćene instance	5
3.3 Parametri algoritama	5
3.4 Rezultati po instancama	5
3.5 Analiza rezultata	6
3.6 Uticaj parametra λ	7
3.7 Grafički prikaz rezultata	7
4 Zaključak	11
Literatura	12

Spisak slika

1	Poređenje GA i $(1 + (\lambda, \lambda))$ GA na instanci st70 (70 gradova)	7
2	Poređenje GA i $(1 + (\lambda, \lambda))$ GA na instanci kroA100 (100 gradova)	8
3	Poređenje GA i $(1 + (\lambda, \lambda))$ GA na instanci kroB150 (150 gradova)	9
4	Poređenje GA i $(1 + (\lambda, \lambda))$ GA na instanci a280 (280 gradova)	10

Spisak tabela

1	Korišćene TSPLIB instance	5
2	Parametri genetskog algoritma	5
3	Poređenje GA i $(1 + (\lambda, \lambda))$ GA na svim instancama	6

1 Uvod

Problem trgovačkog putnika (eng. *Traveling Salesman Problem*) je jedan od najpoznatijih kombinatornih optimizacionih problema. Zadana je mapa sa n gradova i rastojanjima između svaka dva grada, a cilj je pronaći najkraću zatvorenu turu koja posećuje svaki grad tačno jednom i vraća se u polazni.

TSP se definiše na potpunom grafu $G = (V, E)$ sa skupom čvorova $V = \{1, \dots, n\}$ i funkcijom rastojanja $d : V \times V \rightarrow \mathbb{R}^+$. Traži se permutacija π skupa V koja minimizuje ukupnu dužinu ture:

$$f(\pi) = \sum_{i=1}^{n-1} d(\pi(i), \pi(i+1)) + d(\pi(n), \pi(1)) \quad (1)$$

TSP je NP-težak problem, što znači da ne postoji poznati algoritam koji garantuje optimalno rešenje u polinomnom vremenu. Zbog toga se u praksi koriste heuristički i metaheuristički pristupi koji pronalaze približno dobra rešenja u razumnom vremenu.

Genetski algoritmi su među najčešće korišćenim metaheuristikama za rešavanje TSP-a. Zasnivaju se na principima evolucije: populacija rešenja se iterativno poboljšava primenom selekcije, ukrštanja i mutacije. Za TSP su razvijeni specijalizovani operatori ukrštanja poput Order Crossover (OX) operatora, koji garantuju da potomak bude validna permutacija.

Sa druge strane, $(1 + (\lambda, \lambda))$ genetički algoritam predstavlja pristup koji kombinuje mutaciju i ukrštanje na specifičan način. Umesto čitave populacije, održava samo jedno rešenje (roditelja). Svaka generacija ima dve faze: u prvoj se od roditelja pravi λ potomaka mutacijom i bira se najbolji (mutacijski pobednik), a u drugoj se taj pobednik ukršta sa originalnim roditeljem λ puta i bira se najbolje dete iz ukrštanja. Ovaj pristup je predložen u radu Doerr, Doerr i Ebel (2015) gde je teorijski dokazano da ukrštanje daje merljivo ubrzanje u odnosu na čistu mutaciju.

Korišćene su instance iz TSPLIB biblioteke, koja predstavlja ustaljeni skup benchmark instanci u literaturi.

2 Opis rešenja

2.1 Funkcija cilja

Rešenje TSP-a predstavljeno je kao permutacija gradova. Tura $\pi = (\pi(1), \pi(2), \dots, \pi(n))$ definiše redosled obilaska gradova, pri čemu je tura zatvorena — nakon poslednjeg grada $\pi(n)$ putnik se vraća u $\pi(1)$.

Funkcija cilja je ukupna dužina ture definisana jednačinom (1), koju oba algoritma minimizuju. Rastojanja su izračunata kao Euklidska rastojanja zaokružena na najbliži ceo broj:

$$d(i, j) = \text{nint} \left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right) \quad (2)$$

gde nint označava zaokruživanje na najbliži ceo broj.

Matrica rastojanja dimenzije $n \times n$ se preračunava jednom pre pokretanja algoritma, tako da svako izračunavanje funkcije cilja zahteva samo n pristupa tabeli, bez ponovnog računanja korena.

2.2 Genetski algoritam

Implementiran je standardni populacijski genetski algoritam sa sledećim komponentama:

Početna populacija se formira nasumičnim generisanjem 100 permutacija gradova. Svaka jedinka se evaluira računanjem dužine ture.

Turnirska selekcija se koristi za izbor roditelja. Iz populacije se nasumično bira $k = 3$ jedinke i najbolja (najkraća tura) se vraća kao roditelj. Ovaj postupak se ponavlja dvaput za izbor oba roditelja.

Ukrštanje redosledom (Order Crossover, OX) koristi se za generisanje potomka iz dva roditelja. Standardno ukrštanje (presecanje i spajanje) ne funkcioniše za permutacije jer bi proizvelo duplikate i propuštene gradove. OX to rešava tako što kopira nasumični segment iz prvog roditelja, a preostale pozicije popunjava gradovima iz drugog roditelja, čuvajući njihov relativni redosled. Rezultat je uvek validna permutacija. Ukrštanje se primenjuje sa verovatnoćom $p_c = 0.9$.

Mutacija inverzijom obrće nasumični segment ture, što je ekvivalentno 2-opt potezu. Primenjuje se sa verovatnoćom $p_m = 0.2$ na svakog potomka.

Elitizam — jedna najbolja jedinka iz tekuće generacije se direktno prenosi u sledeću, čime se garantuje da se najbolje pronađeno rešenje nikada ne izgubi.

2.3 $(1 + (\lambda, \lambda))$ genetički algoritam

$(1 + (\lambda, \lambda))$ GA u svakom trenutku drži samo jedno rešenje — roditelja. Za razliku od prostijeg $(1 + \lambda)$ EA koji koristi samo mutaciju, ovaj algoritam ima dve faze po generaciji:

Faza mutacije: od roditelja x se napravi λ potomaka, svaki primenom mutacije inverzijom. Pronađe se najbolji potomak — mutacijski pobednik x' . Ova faza troši λ evaluacija.

Faza ukrštanja: originalni roditelj x se ukrsti sa mutacijskim pobednikom x' ukupno λ puta. Pronađe se najbolje dete iz ukrštanja y . Ova faza troši dodatnih λ evaluacija.

Selekcija: ako je y jednako dobar ili bolji od roditelja ($f(y) \leq f(x)$), zamenjuje ga. U suprotnom, roditelj opstaje nepromenjen ("plus" selekcija).

Ideja algoritma je da mutacija istražuje okolinu trenutnog rešenja i pronalazi obećavajući pravac poboljšanja, a ukrštanje zatim kombinuje taj pravac sa originalnim roditeljem, čime se očuvavaju dobre osobine roditelja dok se usvajaju poboljšanja iz mutacije. Svaka generacija troši ukupno 2λ evaluacija.

Korišćenje $\leq$ umesto strogog $<$ u selekciji omogućava kretanje po platou jednako dobrih rešenja, čime se izbegava zaglavljivanje na ravnim delovima prostora pretrage.

2.4 Pošteno poređenje

Oba algoritma su ograničena na isti budžet evaluacija funkcije cilja ($B = 100\,000$). Jedna evaluacija je jedno izračunavanje dužine ture. Ovo je ključno za pošteno poređenje jer $(1 + (\lambda, \lambda))$ GA troši 2λ evaluacija po generaciji, dok GA troši približno jednu evaluaciju po novom potomku. Poređenje po broju generacija bilo bi nepošteno jer jedna generacija jednog i drugog algoritma ne predstavljaju isti računski napor.

3 Eksperimentalni rezultati

3.1 Eksperimentalno okruženje

Algoritmi su implementirani u programskom jeziku Python (verzija 3.12), na operativnom sistemu Ubuntu Linux. Svaki algoritam je pokrenut 30 puta po instanci radi prikupljanja statistike. Budžet evaluacija je $B = 100\,000$ po pokretanju.

3.2 Korišćene instance

Korišćene su četiri instance iz TSPLIB biblioteke, odabrane tako da pokriju raspon veličina od male do velike:

Tabela 1: Korišćene TSPLIB instance

Instanca	Broj gradova	Poznati optimum	Tip rastojanja
st70	70	675	EUC_2D
kroA100	100	21 282	EUC_2D
kroB150	150	26 130	EUC_2D
a280	280	2 579	EUC_2D

3.3 Parametri algoritama

Tabela 2: Parametri genetskog algoritma

Parametar	Vrednost
Veličina populacije	100
Verovatnoća ukrštanja (p_c)	0.9
Verovatnoća mutacije (p_m)	0.2
Veličina turnira (k)	3
Broj elita	1
Budžet evaluacija	100 000

Za $(1+(\lambda, \lambda))$ GA ispitane su vrednosti $\lambda \in \{1, 5, 10, 20, 50\}$, sa istim budžetom evaluacija. Mutacija je inverzija segmenta, a ukrštanje je Order Crossover (OX).

3.4 Rezultati po instancama

U tabeli 3 prikazani su sumirani rezultati za sve instance. Za svaki algoritam navode se najbolja pronađena dužina ture (Best), prosečna dužina preko 30 pokretanja (Avg), standardna devijacija (Std) i procentualno odstupanje proseka od poznatog optimuma (Gap).

Tabela 3: Poređenje GA i $(1 + (\lambda, \lambda))$ GA na svim instancama

Instanca	Algoritam	Best	Avg	Std	Gap (%)
st70	GA	686	723.7	22.7	7.21
	$(1 + (1, 1))$	692	723.6	17.9	7.20
	$(1 + (5, 5))$	681	710.9	15.5	5.32
	$(1 + (10, 10))$	686	714.3	18.9	5.82
	$(1 + (20, 20))$	684	712.9	16.6	5.62
	$(1 + (50, 50))$	677	709.1	18.8	5.05
kroA100	GA	23 231	25 016	947	17.55
	$(1 + (1, 1))$	22 194	23 374	756	9.83
	$(1 + (5, 5))$	21 440	23 263	1021	9.31
	$(1 + (10, 10))$	22 012	22 976	788	7.96
	$(1 + (20, 20))$	21 296	23 031	878	8.22
	$(1 + (50, 50))$	21 709	23 091	781	8.50
kroB150	GA	36 248	41 116	2503	57.35
	$(1 + (1, 1))$	29 220	30 922	798	18.34
	$(1 + (5, 5))$	27 847	29 050	659	11.18
	$(1 + (10, 10))$	27 294	29 082	741	11.30
	$(1 + (20, 20))$	27 853	28 935	668	10.74
	$(1 + (50, 50))$	27 755	29 202	722	11.76
a280	GA	7 655	8 864	448	243.71
	$(1 + (1, 1))$	4 007	4 286	149	66.20
	$(1 + (5, 5))$	3 471	3 636	106	40.97
	$(1 + (10, 10))$	3 411	3 553	92	37.76
	$(1 + (20, 20))$	3 433	3 592	101	39.27
	$(1 + (50, 50))$	3 609	3 759	72	45.75

3.5 Analiza rezultata

Na maloj instanci (st70), $(1 + (\lambda, \lambda))$ GA daje blago bolje rezultate od standardnog GA. GA postiže gap od 7.21%, dok $(1 + (\lambda, \lambda))$ varijante postižu gap oko 5–7%, pri čemu je $\lambda = 50$ najbolji sa gapom 5.05%. Razlike su relativno male jer je 70 gradova dovoljno mali prostor pretrage da oba pristupa konvergiraju ka sličnom kvalitetu.

Na srednjim instancama (kroA100 i kroB150), $(1 + (\lambda, \lambda))$ GA postaje jasno bolji. Na kroA100, GA postiže gap od 17.55%, dok $(1 + (\lambda, \lambda))$ varijante postižu gap oko 8–10%, sa $\lambda = 10$ kao najboljim (7.96%). Na kroB150 je razlika još izraženija: GA postiže gap od 57.35%, dok $(1 + (\lambda, \lambda))$ GA postiže oko 10–18% u zavisnosti od λ .

Na velikoj instanci (a280), razlika je dramatična. GA postiže prosečnu dužinu od 8 864 (gap 243.71%), dok $(1 + (\lambda, \lambda))$ GA sa $\lambda = 10$ postiže prosek 3 553 (gap 37.76%). GA je praktično neupotrebljiv na ovoj veličini sa datim budžetom — njegova populacija od 100 jedinki troši evaluacije na mnogo nasumičnih jedinki bez dovoljnog broja generacija za konvergenciju.

Objašnjenje: GA troši evaluacije na celu populaciju, dok $(1 + (\lambda, \lambda))$ GA fokusira sve evaluacije na poboljšanje jednog rešenja u dva koraka — mutacija istražuje okolinu, a ukrštanje kombinuje pronađeno poboljšanje sa originalnim roditeljem. Pri ograničenom budžetu ova strategija je efikasnija, posebno na većim instancama.

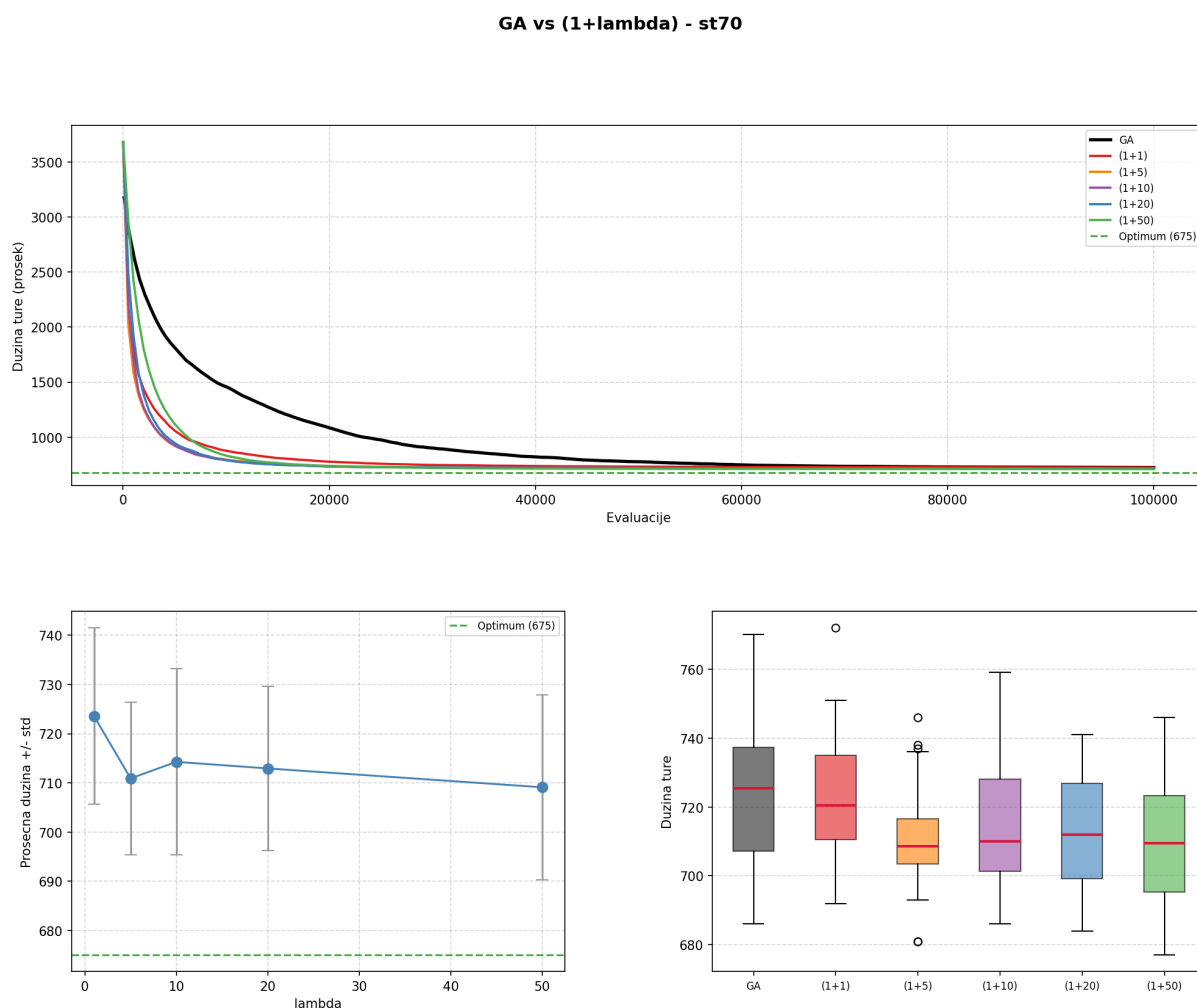
3.6 Uticaj parametra λ

Na graficima (dole levo na svakoj slici) prikazan je uticaj parametra λ na prosečnu dužinu ture. Za razliku od prostijeg $(1+\lambda)$ EA gde je najmanji λ uvek bio najbolji, kod $(1+(\lambda, \lambda))$ GA se uočava optimalna vrednost λ u sredini raspona.

Na manjim instancama (st70) razlike su male, ali veći λ blago pomaže. Na srednjim instancama (kroA100, kroB150) optimalni λ je oko 10–20. Na velikoj instanci (a280) jasno se vidi oblik slova “U”: $\lambda = 1$ daje gap 66.20%, $\lambda = 10$ daje najmanji gap 37.76%, a $\lambda = 50$ ponovo raste na 45.75%.

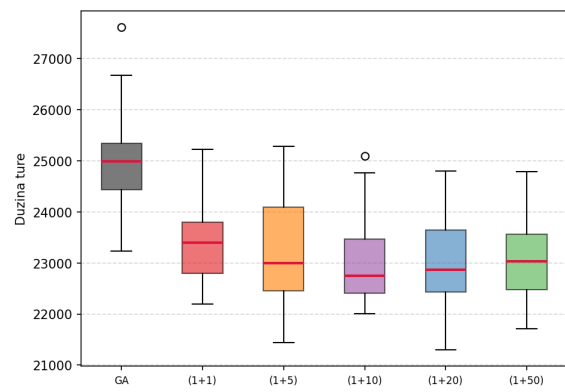
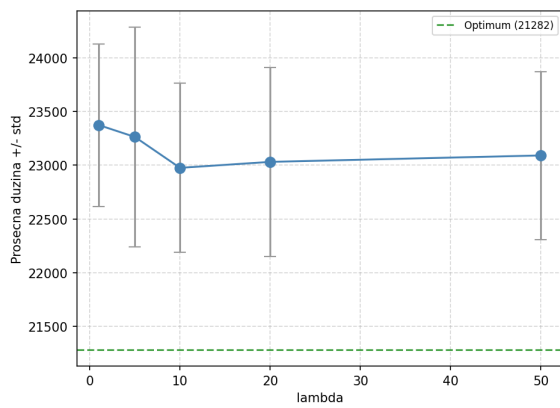
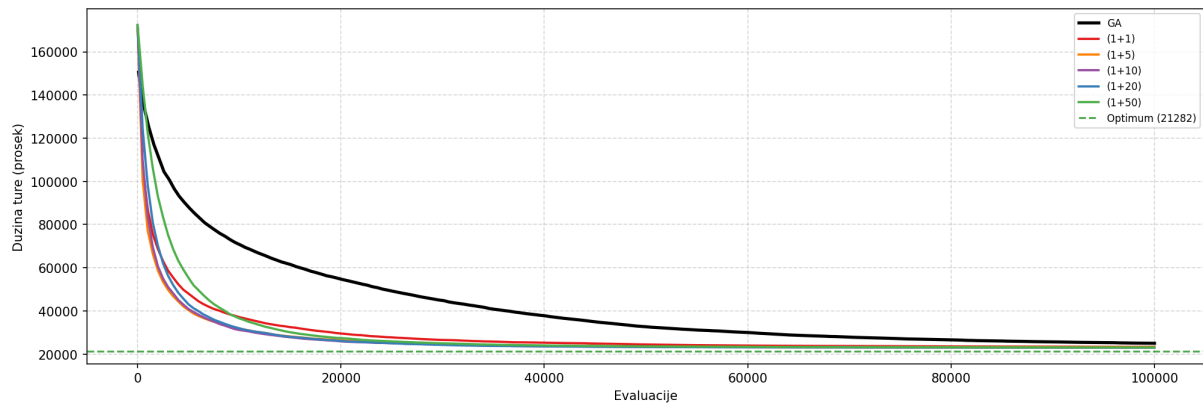
Objašnjenje: premali λ (npr. 1) znači da su obe faze — mutacija i ukrštanje — ograničene na po jednog kandidata, čime se gubi prednost dvofazne pretrage. Preveliki λ (npr. 50) troši $2 \times 50 = 100$ evaluacija po generaciji, pa ostaje samo 1 000 generacija u celom budžetu, što nije dovoljno koraka za konvergenciju na velikim instancama. Umereni λ (5–20) pruža najbolji kompromis: dovoljno kandidata u svakoj fazi za kvalitetan izbor, a istovremeno dovoljno generacija za napredovanje.

3.7 Grafički prikaz rezultata



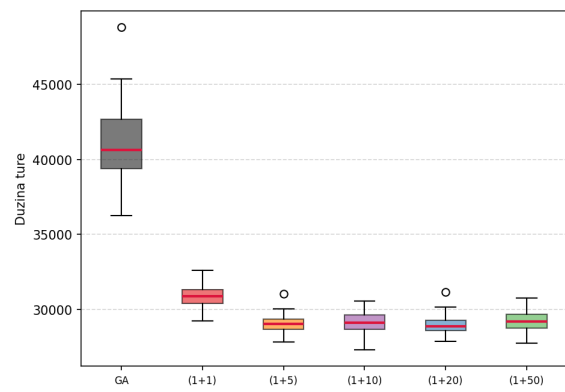
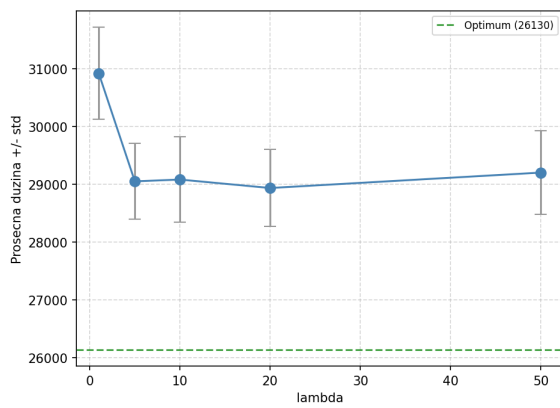
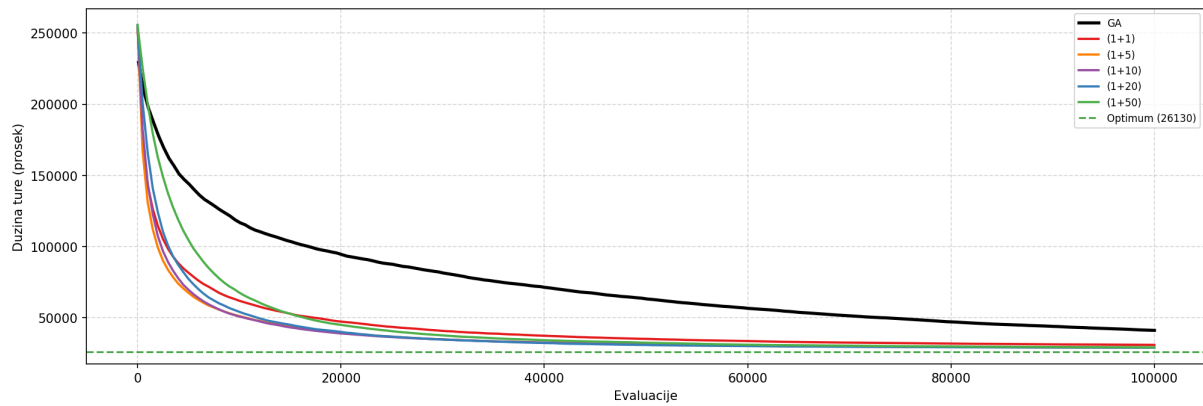
Slika 1: Poređenje GA i $(1 + (\lambda, \lambda))$ GA na instanci st70 (70 gradova)

GA vs (1+lambda) - kroA100



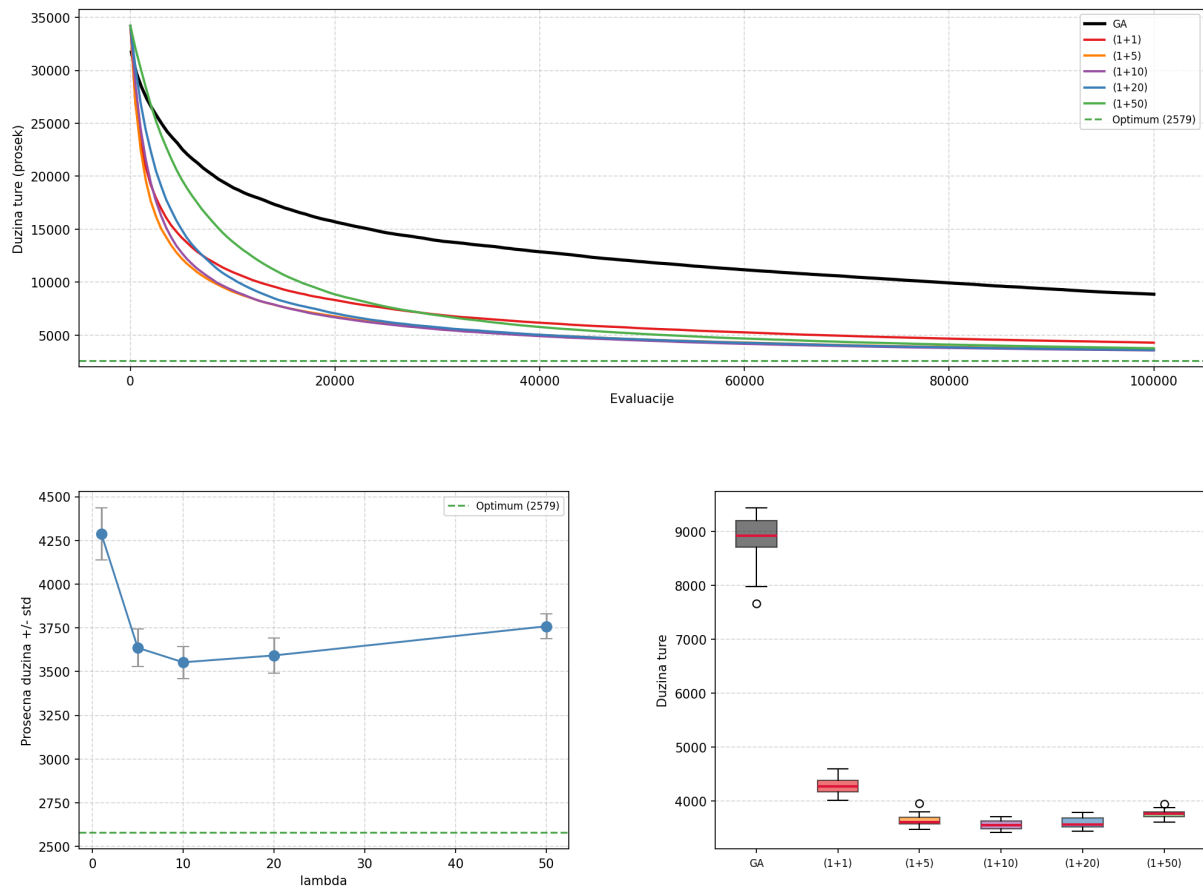
Slika 2: Poređenje GA i $(1 + (\lambda, \lambda))$ GA na instanci kroA100 (100 gradova)

GA vs (1+lambda) - kroB150



Slika 3: Poređenje GA i $(1 + (\lambda, \lambda))$ GA na instanci kroB150 (150 gradova)

GA vs (1+lambda) - a280



Slika 4: Poređenje GA i $(1 + (\lambda, \lambda))$ GA na instanci a280 (280 gradova)

Na graficima konvergencije (gore) uočava se da $(1 + (\lambda, \lambda))$ GA brže konvergira u ranim fazama pretrage, dok GA sporije ali stabilnije opada. Na a280 je razlika posebno upečatljiva — GA kriva ostaje daleko iznad $(1 + (\lambda, \lambda))$ krivih tokom celog budžeta.

Box-plot dijagrami (dole desno) vizuelno prikazuju raspodelu finalnih rešenja. Na a280, GA kutija je potpuno odvojena od $(1 + (\lambda, \lambda))$ kutija, što potvrđuje da razlika nije posledica pojedinačnih outlajera već sistematska. Takođe se na box-plotu vidi da je $\lambda = 1$ primetno lošiji od ostalih λ vrednosti, što potvrđuje da dvofazna pretraga zahteva više od jednog kandidata po fazi da bi bila efikasna.

4 Zaključak

U ovom radu implementirana su i eksperimentalno poređena dva pristupa za rešavanje TSP-a: standardni populacijski genetski algoritam i $(1 + (\lambda, \lambda))$ genetički algoritam.

Rezultati su pokazali da se performanse dva pristupa značajno razlikuju u zavisnosti od veličine instance. Na maloj instanci (70 gradova) oba pristupa postižu slične rezultate. Međutim, sa porastom broja gradova, $(1 + (\lambda, \lambda))$ GA postaje sve bolji u odnosu na standardni GA, da bi na instanci od 280 gradova razlika bila dramatična — $(1 + (\lambda, \lambda))$ GA sa $\lambda = 10$ postiže gap od oko 38% naspram GA sa gapom od preko 240%. Ovo ukazuje na to da pri fiksnom budžetu evaluacija, strategija dvofaznog poboljšanja jednog rešenja (mutacija pa ukrštanje) može biti znatno efikasnija od populacijskog pristupa, posebno na velikim instancama.

Analiza uticaja parametra λ pokazala je da postoji optimalna vrednost u sredini ispitivanog raspona, tipično oko $\lambda = 10$ –20. Ovo je razlika u odnosu na prostiji $(1 + \lambda)$ EA gde manji λ obično daje bolje rezultate. Kod $(1 + (\lambda, \lambda))$ GA, faza ukrštanja zahteva kvalitetnog mutacijskog pobednika, za čije pronalaženje je potrebno više kandidata, pa premali λ poništava prednost dvofazne pretrage.

Treba naglasiti da se rezultati odnose na konkretan budžet od 100 000 evaluacija — sa znatno većim budžetom, odnos snaga bi se mogao promeniti u korist GA jer bi populacijski pristup imao dovoljno generacija za konvergenciju. Takođe, nijedan od implementiranih pristupa ne dostiže kvalitet specijalizovanih TSP heuristika iz literature (poput LKH algoritma), ali to nije ni bio cilj rada. Cilj je bio pošteno poređenje dva opšta evolutivna pristupa na istom problemu.

Pravci daljeg istraživanja uključuju: povećanje budžeta evaluacija kako bi se ispitalo ponašanje GA na većim budžetima, ispitivanje adaptivnog podešavanja parametra λ tokom pretrage (kao što je predloženo u originalnom radu), kao i hibridizacija sa lokalnom pretragom poput 2-opt.

Literatura

- [1] Doerr B., Doerr C., Ebel F. 2015. From black-box complexity to designing new genetic algorithms. *Theoretical Computer Science*, 567: 87–104.
<https://doi.org/10.1016/j.tcs.2014.11.028>
- [2] Bassin A., Buzdalov M. 2020. The $(1 + (\lambda, \lambda))$ genetic algorithm for permutations. U: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion (GECCO '20)*. New York: ACM, 1669–1677.
<https://doi.org/10.1145/3377929.3398148>
- [3] Reinelt G. 1991. TSPLIB — A Traveling Salesman Problem Library. *ORSA Journal on Computing*, 3(4): 376–384.
<https://doi.org/10.1287/ijoc.3.4.376>
- [4] TSPLIB95 — Benchmark instance za simetrični problem trgovačkog putnika. Universität Heidelberg. (Pristupljeno 2026.)
<https://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp.html>